

2.1 ФОРМИ ЗАПИСУ ЗЛП

Залежно від вигляду системи обмежень, виділяють дві основні форми.

1. Стандартна (симетрична) форма

1. Стандартна (симетрична) форма

Це найбільш природний вигляд задачі, коли ми тільки-но переклали її з тексту в математику. Суть у тому, що всі обмеження записані у вигляді **нерівностей**.

- **Для задачі на максимум (max):** усі обмеження мають знак « $\leq$ » (менше або дорівнює). Це логічно: ми хочемо отримати максимум результату, маючи обмежені ресурси (ми не можемо витратити більше, ніж є).
- **Для задачі на мінімум (min):** усі обмеження мають знак « $\geq$ » (більше або дорівнює). Наприклад, ми мінімізуємо витрати, але маємо виконати план не гірше певного мінімуму.
- Усі змінні $x_i \geq 0$.

2. Канонічна (основна) форма

Це найважливіша форма для машинних обчислень. Саме з нею працює симплекс-метод та більшість програмних солверів.

Її головна відмінність: **усі обмеження є строгими рівняннями (знак « $=$ »)**, крім умови невід'ємності змінних ($x_i \geq 0$).

З погляду програмування, перехід до рівнянь дозволяє записати систему обмежень у вигляді матриці $A \times x = b$ і застосовувати до неї методи лінійної алгебри (наприклад, метод Гауса), перебираючи базиси матриці.

Як звести задачу до канонічної форми (Правила переходу)

Як звести задачу до канонічної форми (Правила переходу)

Оскільки на практиці задачі майже завжди формуються з нерівностями, потрібно вміти перетворювати їх на рівняння. Це робиться шляхом введення **додаткових (балансових) змінних**.

Правило 1: Перетворення нерівності типу « $\leq$ »

Якщо маємо обмеження $2x_1 + 3x_2 \leq 10$, це означає, що ліва частина менша за праву на якусь величину. Щоб зробити з цього рівняння, ми додаємо невід'ємну балансову змінну $x_3 \geq 0$.

- Було: $2x_1 + 3x_2 \leq 10$
- Стало: $2x_1 + 3x_2 + x_3 = 10$
(Економічний зміст: x_3 — це залишок невикористаного ресурсу).

Правило 2: Перетворення нерівності типу « $\geq$ »

Якщо маємо обмеження $5x_1 + x_2 \geq 15$, ліва частина більша за праву. Щоб урівняти, ми віднімаємо додаткову змінну $x_4 \geq 0$.

- Було: $5x_1 + x_2 \geq 15$

- Стало: $5x_1 + x_2 - x_4 = 15$

(Економічний зміст: x_4 — це перевиконання мінімального плану).

Правило 3: Зміна напрямку оптимізації

Якщо алгоритм налаштований шукати тільки максимум, а нам треба знайти мінімум функції Z , ми просто множимо цільову функцію на -1 і шукаємо максимум.

- $\min Z = 2x_1 + 4x_2 \implies \max(-Z) = -2x_1 - 4x_2$

Правило 4: Змінні без обмеження на знак

Якщо в задачі якась змінна x_1 може бути і додатною, і від'ємною (а для алгоритму всі мають бути ≥ 0), ми замінюємо її різницею двох нових невід'ємних змінних:

$$x_1 = x'_1 - x''_1, \text{ де } x'_1 \geq 0, x''_1 \geq 0.$$

2.2 Графічний метод розв'язання ЗЛП

Графічний метод — це візуальний спосіб знайти оптимальне рішення. Його головне обмеження полягає в тому, що він **працює лише для задач із двома змінними** (x_1 та x_2), оскільки ми малюємо їх на двовимірній площині. (Для трьох змінних потрібен 3D-графік, що складно малювати на папері, а для 4 і більше — метод взагалі не підходить).

Алгоритм графічного методу (крок за кроком):

Крок 1. Побудова системи координат.

Оскільки зазвичай діє умова невід'ємності ($x_1 \geq 0, x_2 \geq 0$), ми працюємо лише в першій чверті декартової системи координат. Вісь X — це x_1 , вісь Y — це x_2 .

Крок 2. Побудова області допустимих розв'язків (ОДР).

Ми беремо нашу систему обмежень (у вигляді нерівностей) і малюємо кожне з них:

1. Тимчасово замінюємо знак нерівності (наприклад, $\leq$) на знак дорівнює ($=$).
Отримуємо рівняння прямої лінії.
2. Знаходимо дві точки (наприклад, перетини з осями, підставивши $x_1 = 0$, а потім $x_2 = 0$) і проводимо через них пряму.
3. Визначаємо, яку півплощину відтинає ця пряма. Для цього беремо контрольну точку (найпростіше $(0; 0)$) і підставляємо в початкову нерівність. Якщо нерівність виконується (наприклад, $0 \leq 24$ — це правда), то штрихуємо область у бік точки $(0; 0)$. Якщо ні — у протилежний.
4. Перетин **усіх** заштрихованих областей від усіх обмежень утворює **Область допустимих розв'язків (ОДР)** — зазвичай це певний багатокутник. Кожна точка всередині нього задовольняє всі обмеження задачі.

Крок 3. Побудова вектора градієнта та лінії цільової функції.

Цільова функція має вигляд $Z = c_1x_1 + c_2x_2$.

1. Будуємо вектор **градієнта** $n = (c_1; c_2)$. Він починається з початку координат $(0; 0)$ і йде в точку з координатами $(c_1; c_2)$. Цей вектор показує напрямок **найшвидшого зростання** функції Z .
2. Будуємо лінію цільової функції. Прирівнюємо Z до якогось зручного числа (або просто до 0: $c_1x_1 + c_2x_2 = 0$) і малюємо цю пряму. Вона завжди буде перпендикулярна до вектора n .

Крок 4. Знаходження оптимуму (рух паралельним перенесенням).

Беремо лінію цільової функції і починаємо рухати її паралельно самій собі:

- Якщо ми шукаємо **максимум**, рухаємо пряму **за напрямком** вектора n .
- Якщо шукаємо **мінімум**, рухаємо **проти напрямку** вектора n .

Рухаємо пряму доти, поки вона не торкнеться **останньої крайньої точки** багатокутника ОДР перед тим, як остаточно вийти за його межі.

Крок 5. Обчислення результату.

Остання точка дотику і є оптимальним розв'язком. Знаходимо її координати $(x_1; x_2)$ (розв'язавши систему з двох рівнянь прямих, які утворили цей кут) і підставляємо їх у цільову функцію Z , щоб знайти максимальний або мінімальний прибуток/витрати.

2.3. Властивості багатогранних множин (опуклість, кутові точки)

2.3. Властивості багатогранних множин (опуклість, кутові точки)

Це теоретичне обґрунтування того, чому графічний метод взагалі працює.

- **Опуклість:** Область допустимих розв'язків завжди є *опуклою множиною*. Опукла множина — це така фігура, де відрізок, що з'єднує будь-які дві точки всередині неї, також повністю лежить всередині неї (простими словами: у фігури немає "вм'ятин").
- **Теорема про кутові точки:** Оптимальне значення лінійної цільової функції завжди досягається в одній із **кутових точок** (вершин) багатогранника розв'язків.

Саме тому ми можемо не перевіряти мільйони точок всередині багатокутника, а дивимися лише на його кути!

Особливі (нестандартні) випадки графічного методу:

1. **Альтернативний оптимум:** Лінія цільової функції паралельна одній зі сторін багатокутника. Тоді останнім дотиком буде не одна точка, а цілий відрізок. Рішень безліч.
2. **Необмеженість функції:** ОДР не замкнена з одного боку (йде в нескінченність), і ми можемо рухати пряму за градієнтом нескінченно довго. Оптимального рішення немає ($Z \rightarrow \infty$).
3. **Несумісність умов:** Обмеження суперечать одне одному (наприклад, $x \leq 2$ і $x \geq 5$). ОДР порожня, розв'язків немає.

2.4. Симплекс-метод

Якщо графічний метод працює лише для двох змінних і ми шукаємо кути візуально, то симплекс-метод — це **алгебраїчний спосіб** робити те саме для будь-якої кількості змінних. Він послідовно "перестрибує" з однієї кутової точки багатогранника рішень в іншу, щоразу покращуючи (або не погіршуючи) значення цільової функції, поки не знайде найкращу.

Щоб метод запрацював, задача обов'язково має бути у **канонічній формі** (система рівнянь).

Алгоритм симплекс-методу (крок за кроком):

Крок 1. Знаходження початкового опорного плану (базису).

Щоб почати "стрибати" по кутах, треба знайти стартову точку. У канонічній системі рівнянь ми вибираємо **базисні змінні** (їхня кількість дорівнює кількості рівнянь), а всі інші змінні робимо **вільними** і прирівнюємо до нуля.

Ідеальний старт — коли в системі рівнянь вже є "одиначна матриця" (змінні, які входять лише в одне рівняння з коефіцієнтом 1). Тоді вони стають базисом.

Крок 2. Побудова симплекс-таблиці.

Усі коефіцієнти задачі заносяться у спеціальну таблицю. Найважливіший у ній — останній (нижній) рядок. Він називається **індексним (або оцінковим) рядком** (позначається зазвичай Δ_j). Він показує, чи можна ще покращити наш план.

Крок 3. Перевірка на оптимальність.

Ми дивимося на індексний рядок:

- Якщо ми шукаємо **максимум**, то план оптимальний, коли в індексному рядку **немає від'ємних чисел** (всі $\Delta_j \geq 0$).
- Якщо є хоча б одне від'ємне число, план не оптимальний, і його можна покращити.

Крок 4. Вибір ведучого стовпця і ведучого рядка (перехід до нової точки).

- **Ведучий стовпець (хто увійде в базис):** вибираємо за найбільшим за модулем від'ємним числом в індексному рядку (для задачі на максимум). Ця змінна принесе найбільший прибуток.
- **Ведучий рядок (хто вийде з базису):** щоб дізнатися, яке обмеження спрацює першим, ми ділимо стовпець вільних членів (ресурси) на додатні елементи ведучого стовпця. Рядок, де це **відношення найменше**, стає ведучим.
- Перетин ведучого рядка і стовпця дає **дозвільний (ведучий) елемент**.

Крок 5. Перерахунок таблиці (крок Жордана-Гауса).

За допомогою математичних перетворень (ділення рядка на ведучий елемент та додавання/віднімання рядків) ми робимо так, щоб на місці ведучого елемента стала 1, а в усьому іншому стовпці — нулі. Ми отримали нову таблицю (перейшли в нову кутову точку). Далі повертаємося до Кроку 3.

2.5. Метод штучного базису (М-метод)

Проблема: Що робити, якщо після переходу до канонічної форми ми не бачимо очевидного початкового базису (одиначної матриці)? Наприклад, у нас були обмеження типу « $\geq$ » або « $=$ ».

Рішення: Застосовують метод штучного базису.

Ми штучно додаємо до таких рівнянь **штучні змінні** ($x_{\text{ш}} \geq 0$), щоб вони стали базисом.

Але щоб ці "вигадані" змінні не зіпсували реальний результат, ми вводимо їх у цільову функцію з величезним **штрафом**:

- Для задачі на **max**: додаємо $-M \cdot x_{\text{ш}}$ (де M — дуже велике число, наприклад, 1000000).
- Для задачі на **min**: додаємо $+M \cdot x_{\text{ш}}$.

Симплекс-метод почне працювати, і через цей гігантський штраф він "злякається" і постарается якнайшвидше вивести ці штучні змінні з базису (зробити їх рівними нулю). Якщо наприкінці розв'язку штучні змінні вийшли з базису, ми знайшли реальний оптимум. Якщо хоч одна штучна змінна залишилась у базисі з додатним значенням — реальна задача не має розв'язку (умови суперечливі).

2.6. Виродженість та антицикліни

- **Виродженість:** Це ситуація, коли одна або кілька базисних змінних стають рівними нулю (графічно це означає, що в одній кутовій точці перетинаються більше прямих, ніж потрібно, наприклад, 3 прямі у 2D-просторі).
- **За циклення:** Через виродженість симплекс-метод може "зламатися". Він почне міняти змінні в базисі, але значення цільової функції не буде змінюватись (бо ми стоїмо в тій самій точці з нульовим кроком). Програма почне ходити по колу (зациклиться).
- **Антицикліни (Правило Бленда):** Щоб вийти з циклу, використовують спеціальні правила вибору ведучого рядка і стовпця. Наприклад, якщо є кілька однакових від'ємних оцінок або однакових мінімальних відношень, завжди обирають змінну з **найменшим індексом** (наприклад, між x_2 і x_4 виберуть x_2). Це гарантує вихід із "мертвої петлі".

2.7. Двоїста задача та двоїстість у лінійному програмуванні

1. Правила побудови двоїстої задачі

Будь-яка задача лінійного програмування має свою двоїсту пару. Щоб математично перетворити пряму задачу на двоїсту, діють строгі правила (ніби матриця віддзеркалюється):

1. **Цільова функція змінює напрямок:** Якщо пряма шукала $\max Z$ (прибуток), то двоїста шукає $\min F$ (витрати на ресурси), і навпаки.
2. **Змінюються місцями «ресурси» і «прибутки»:** * Коефіцієнти прибутку з цільової функції прямої задачі ($c_1, c_2 \dots$) стають стовпцем вільних членів (обмеженнями) у двоїстій.
 - Запаси ресурсів ($b_1, b_2 \dots$) стають коефіцієнтами нової цільової функції.
3. **Матриця коефіцієнтів транспонується:** Рядки системи обмежень прямої задачі стають стовпцями у двоїстій. (Наприклад, якщо в першому рівнянні були коефіцієнти $2x_1 + 3x_2$, то у двоїстій вони стануть $2y_1$ та $3y_1$ у різних рівняннях).
4. **Змінюються знаки нерівностей:**
 - Обмеження « $\leq$ » (менше-рівне) у прямій задачі $\implies$ змінна $y_i \geq 0$ у двоїстій.
 - Строге рівняння « $=$ » у прямій $\implies$ змінна y_i може мати будь-який знак (і додатний, і від'ємний).

2. Економічний зміст змінних (Дуже важливо для колоквіуму!)

У прямій задачі змінні x_1, x_2 — це кількість продукції, яку треба випустити.

У двоїстій задачі змінні y_1, y_2 — це ціни (їх ще називають *тіньовими цінами* або *двоїстими оцінками*) кожного виду ресурсу.

Ці оцінки показують, **наскільки збільшиться наш максимальний прибуток, якщо ми десь дістанемо ще одну додаткову одиницю цього ресурсу.**

3. Основні теореми двоїстої задачі

Це фундамент, який пояснює, навіщо ми це все рахуємо.

Перша теорема двоїстості (про збіг екстремумів):

Якщо одна із задач має оптимальний розв'язок, то й інша його має. При цьому максимальний прибуток прямої задачі точно дорівнює мінімальним витратам двоїстої:

$$\max Z = \min F$$

(Тобто, якщо ти знайшов прибуток 1000 доларів у прямій задачі, то загальна вартість усіх твоїх ресурсів за тіньовими цінами теж складе 1000 доларів).

Друга теорема двоїстості (Умови доповнювальності нежорсткості):

Це правило про дефіцитність ресурсів.

- Якщо в оптимальному плані якийсь ресурс **не використано повністю** (залишився на складі, тобто нерівність виконалась як строга $<$), то його двоїста оцінка дорівнює **нулю** ($y_i = 0$). Логічно: навіщо платити за ресурс, якого і так в надлишку? Його цінність нульова.
- Якщо ж ресурс використано **повністю** (під нуль), то він є дефіцитним, і його двоїста оцінка буде **більшою за нуль** ($y_i > 0$).

2.8. Двоїстий симплексний метод

4. Двоїстий симплекс-метод

Звичайний симплекс-метод починається з того, що план є допустимим (усі ресурси ≥ 0), але неоптимальним.

Двоїстий симплекс-метод застосовують тоді, коли ми маємо план, який виглядає **оптимальним** (в індексному рядку немає від'ємних оцінок, ніби все ідеально), але він **недопустимий** фізично — у стовпці вільних членів (ресурсів) з'явилися **від'ємні числа** (ніби ми влізли в борг по матеріалах).

Як він працює:

1. Замість того, щоб шукати найкращий стовпець, ми спочатку шукаємо **ведучий рядок**: вибираємо найбільше за модулем від'ємне число у стовпці ресурсів.
2. Потім шукаємо **ведучий стовпець**: ділимо оцінки з індексного рядка на відповідні від'ємні елементи ведучого рядка і беремо найменше за модулем значення.
3. Робимо перерахунок (як у звичайному методі Гауса-Жордана), поки всі числа у стовпці ресурсів не стануть додатними.

3. Цілочисельне лінійне програмування (ЦЛП)

Навіщо це потрібно?

Уяви, що ти розрахував оптимальний план для заводу, і звичайний симплекс-метод видає: "Щоб отримати максимальний прибуток, треба випустити 2.7 тракторів і 4.2 комбайни".

Але ж ми не можемо випустити 0.7 трактора! Нам потрібні строго **цілі числа**.

Здавалося б, найпростіше рішення — просто округлити (зробити 3 трактори і 4 комбайни).

Але математично це дуже небезпечно:

1. Округлений план може вийти за межі наявних ресурсів (на 3-й трактор банально не вистачить металу, умова перестане виконуватись).
2. Округлений план може виявитися далеко не найоптимальнішим (іноді краще виробити 2 трактори і 5 комбайнів).

Тому придумали спеціальні методи, і перший з них — **Метод Гоморі** (або метод відтинаючих площин).

3.1. Метод Гоморі

Суть методу (графічно):

Ми беремо нашу фігуру ОДР (область допустимих розв'язків) і знаходимо її звичайний "дробовий" максимум у кутовому розв'язку. Потім ми беремо віртуальний "ніж" і **відрізаємо** цей дробовий кут від фігури так, щоб нова лінія зрізу пройшла максимально близько до найближчих цілочисельних точок, але **не відрізала жодну цілочисельну точку** всередині фігури. Ми робимо такі відрізання доти, поки новий кут фігури не стане ідеально цілим.

Крок 1. Розв'язок без цілочисельності.

Ми тимчасово забуваємо про те, що змінні мають бути цілими. Розв'язуємо задачу звичайним симплекс-методом до кінця. Отримуємо оптимальну таблицю.

Крок 2. Перевірка на цілочисельність.

Дивимось на стовпець відповідей (вільних членів b_i):

- Якщо всі значення в базисі — цілі числа, бінго! Задача розв'язана, це і є цілочисельний оптимум.
- Якщо є хоча б одне дробове число (наприклад, $x_1 = 3.4$) — переходимо до Кроку 3.

Крок 3. Вибір рядка для "відрізання".

Якщо дробових відповідей кілька, вибираємо ту, у якої **найбільша дробова частина**.

(Приклад: якщо є відповіді 2.1 і 4.8, ми вибираємо рядок із 4.8, бо його дробова частина 0.8 більша за 0.1. Вважається, що так ми відріжемо "найбільший шматок" фігури і швидше

Крок 4. Побудова додаткового обмеження (відрізання Гоморі).

Ми беремо всі елементи обраного рядка з симплекс-таблиці (коефіцієнти перед змінними і саму відповідь) і **відкидаємо їх цілі частини**, залишаючи тільки дробові.

З цих дробових частин ми формуємо нове рівняння (з новою балансовою змінною $S \geq 0$).

Головна фішка: у цьому новому рівнянні у стовпці відповідей (b_i) з'явиться **від'ємне число** (мінус дробова частина вільного члена).

Ми дописуємо це нове рівняння як **додатковий рядок** униз нашої симплекс-таблиці (і додаємо новий стовпець для змінної S).

Крок 5. Застосування Двоїстого симплекс-методу.

Зверни увагу на те, що сталося з таблицею!

- У нижньому (індексному) рядку немає від'ємних чисел (тобто план нібито оптимальний).
- Але у стовпці відповідей з'явився "мінус" (тобто план став недопустимим фізично — ми ж ніби відрізали шматок ОДР).

Саме тому тут ідеально працює **Двоїстий симплексний метод**, який ми обговорювали раніше. Ми робимо крок двоїстим симплексом: вибираємо цей новий рядок як ведучий (бо там мінус), знаходимо ведучий стовпець, робимо перерахунок.

Крок 6. Повторення (Цикл).

Після перерахунку знову дивимося на стовпець відповідей. Всі числа стали цілими?

Кінець. Знову з'явилися дробби? Робимо нове відрізання (повертаємося до Кроку 3).

Крок 1. Початковий розв'язок (корінь дерева).

Як і в Гоморі, ми спочатку "забуваємо" про вимогу цілочисельності і розв'язуємо задачу звичайним симплекс-методом.

- Отримуємо результат. Якщо всі змінні вийшли цілими — супер, задача розв'язана.
- Якщо ж є дробі (наприклад, $Z = 50.5$, при цьому $x_1 = 3.7$), ми розуміємо: 50.5 — це наш абсолютний теоретичний максимум (стеля). Справжній цілочисельний прибуток точно буде меншим або рівним цьому числу.

Крок 2. Розгалуження (Гілки).

Ми бачимо, що $x_1 = 3.7$. Але тракторів не може бути 3.7! Їх може бути або 3 і менше, або 4 і більше. Між 3 і 4 — порожнеча, там немає цілих чисел.

Тому ми **розбиваємо нашу задачу на дві нові підзадачі**:

- **Гілка 1 (Ліва):** Беремо нашу початкову задачу і додаємо до неї нове обмеження:
$$x_1 \leq 3.$$
- **Гілка 2 (Права):** Беремо нашу початкову задачу і додаємо інше обмеження: $x_1 \geq 4$.

Таким чином ми фізично "вирізуали" з області допустимих розв'язків ту частину, де x_1 знаходиться від 3 до 4.

Крок 4. Продовження.

Якщо гілка має дробовий розв'язок і її прибуток **більший** за наш поточний рекорд (наприклад, $Z = 48.2$), це означає, що там потенційно може ховатися кращий цілочисельний розв'язок (наприклад, 47). Тоді ми беремо змінну з дробом із цієї гілки і **знову розгалужуємо** її на дві нові підзадачі.

Алгоритм закінчується тоді, коли **всі гілки дерева відсічені**. Той "рекорд", який залишився на кінець алгоритму, і є ідеальним оптимальним розв'язком задачі.

Чому цей метод крутий?

- Він дуже логічний. Ми просто систематично перебираємо варіанти, але відкидаємо завідомо "мертві" зони.
- Якщо задача розв'язується графічно (2 змінні), то розгалуження виглядає як проведення вертикальних чи горизонтальних ліній, що "розрізають" багатокутник ОДР на дві менші частини.

Крок 3. Знаходження меж і перевірка (Межі).

Ми розв'язуємо ці дві нові підзадачі (зазвичай графічно або симплекс-методом). Далі дивимося на результати кожної гілки.

Тут вступають в дію **правила відсікання**. Ми закриваємо (відсікаємо) гілку і більше її не розгалужуємо в трьох випадках:

1. **Гілка порожня (немає розв'язків):** Наприклад, ми додали умову $x_1 \geq 4$, а ресурсів вистачає максимум на 3.5. Задача несумісна. Відсікаємо гілку.
2. **Знайдено цілочисельний розв'язок:** Наприклад, у лівій гілці ми отримали $x_1 = 3$, $x_2 = 2$ і прибуток $Z = 45$. Це повністю цілий план! Ми записуємо $Z = 45$ як наш **РЕКОРД**. Гілку далі не ділимо, але запам'ятовуємо цей результат.
3. **Межа гілки гірша за наш рекорд:** Припустимо, ми розв'язали праву гілку (з дробами) і отримали теоретичний максимум $Z = 42.4$. Оскільки це максимум, то навіть якщо ми знайдемо там цілі числа далі по гілці, прибуток буде ще меншим (наприклад, 41 або 40). А в нас уже є в кишені гарантований "рекорд" $Z = 45$ з лівої гілки! Тому праву гілку немає сенсу досліджувати — відсікаємо її.

3.3 Задача комівояжера

Крок 1. "Зведення" (Шукаємо гарантований мінімум витрат)

Ми хочемо знайти мінімальну суму, яку по-любому доведеться витратити, щоб просто виїхати з кожного міста і заїхати в кожне.

1. Ти дивишся на перший рядок (ціни виїзду з 1-го міста). Найменший квиток коштує 10 грн. Ти кажеш: "Я точно витрачу мінімум 10 грн, щоб виїхати звідси". Ти **віднімаєш** ці 10 грн від усіх цін у цьому рядку, а десятку відкладаєш у "скарбничку" (це твоя нижня межа H). Тепер найдешевший квиток тут коштує 0.
2. Робиш так для кожного рядка.
3. Потім дивишся на стовпці (ціни в'їзду). Якщо десь ще немає нуля, ти знаходиш там мінімум, віднімаєш його від усього стовпця і теж кидаєш у скарбничку.

Результат: Тепер у кожному рядку і стовпці є хоча б один 0. Цей нуль означає "найдешевший, умовно безкоштовний шлях". А в скарбничці лежить сума H — це наш стартовий рекорд. Дешевше за цю суму об'їхати міста фізично неможливо.

Крок 2. "Вартість відмови" (Оцінка нулів)

Тепер перед нами стоїть вибір: якими з цих "безкоштовних" нулів скористатися? Ми не можемо взяти всі нулі, бо деякі можуть вести в те саме місто або створювати підцикли. Ми маємо вибрати один нуль, за який будемо "триматися". Як його обрати? Ми рахуємо штраф для кожного нуля: **"Що буде, якщо я відмовлюся від цього квитка за 0?"**

- Якщо ти відмовляєшся від нуля, тобі доведеться купувати **наступний найдешевший квиток** у цьому ж рядку (щоб якось виїхати) і в цьому ж стовпці (щоб якось заїхати).
- **Формула штрафу:** Штраф нуля = (мінімум серед інших чисел у його рядку) + (мінімум серед інших чисел у його стовпці).

Ми рахуємо ці штрафи для всіх нулів у таблиці.

Крок 3. Розгалуження (Два всесвіти)

Ми знаходимо нуль із **найбільшим штрафом**. Це означає: відмовлятися від цього маршруту (наприклад, з міста 2 у місто 4) найневигідніше, бо ми втратимо купу грошей. Тут ми ділимо нашу задачу на дві гілки:

Гілка 1: Ми беремо цей маршрут ($2 \rightarrow 4$)

- Ми кажемо: "Все, ми точно їдемо з 2 в 4".
- Ми **викреслюємо** з таблиці рядок 2 (бо з міста 2 ми вже виїхали) і стовпець 4 (бо в місто 4 ми вже заїхали). Таблиця стає меншою на один розмір!
- Щоб випадково не поїхати назад, ми ставимо ∞ у клітинку з маршрутом $4 \rightarrow 2$.
- Нижня межа (сума в скарбничці) для цієї гілки поки залишається такою ж.

Гілка 2: Ми відмовляємося від маршруту ($2 \rightarrow 4$)

- Ми кажемо: "Якимось дивом ми сюди не поїдемо".
- Тоді ми ставимо ∞ у клітинку $2 \rightarrow 4$, щоб назавжди заборонити цей шлях.
- Сума в нашій скарбничці для цієї гілки **збільшується на розмір штрафу** (бо ми обрали дорожчий шлях).

Крок 4. Відсікання

Ми порівнюємо скарбнички. Якщо скарбничка Гілки 2 стала дуже дорогою (дорожчою за вже відомі маршрути), ми її просто "відсікаємо" і більше не розглядаємо.

А зменшену таблицю з Гілки 1 ми знову починаємо "зводити" (Крок 1), знову рахуємо нулі і так відкушуємо по одному маршруту, поки таблиця не зникне.

4. Транспортна задача (ТЗ)

Закрита та відкрита моделі ТЗ

1. Закрита (збалансована) модель

Це ідеальна ситуація, коли сумарний запас усіх постачальників точно дорівнює сумарній потребі всіх споживачів:

$$\sum_{i=1}^m a_i = \sum_{j=1}^n b_j$$

- **Особливість:** Така задача завжди має допустимий розв'язок. Систему обмежень можна записувати через строгі рівняння (=).

2. Відкрита (незбалансована) модель

У реальному житті баланс буває рідко. Якщо сумарний запас не дорівнює сумарному попиту ($\sum a_i \neq \sum b_j$), модель називається **відкритою**.

Алгоритми ТЗ (наприклад, метод мінімального елемента чи метод потенціалів) фізично не вміють працювати з відкритою моделлю. Тому її **примусово зводять до закритої** за допомогою введення фіктивних пунктів.

Є два варіанти дисбалансу:

- **Варіант А: Запаси більші за потреби ($\sum a_i > \sum b_j$)**

Товару на складах більше, ніж хочуть купити магазини. На складах утворюється надлишок.

- **Рішення:** Ми створюємо **фіктивного споживача** B_{n+1} (додатковий стовпець у таблиці).
- **Потреба фіктивного споживача:** $b_{n+1} = \sum a_i - \sum b_j$ (він "купує" весь надлишок товару, щоб урівняти баланс).
- **Тарифи:** Вартість перевезення до цього фіктивного споживача з будь-якого складу ставиться рівною **нулю** ($c_{i,n+1} = 0$), оскільки насправді товар нікуди не везуть, він просто залишається лежати на складі.

- **Варіант Б: Запаси менші за потреби ($\sum a_i < \sum b_j$)**

Магазини хочуть більше товару, ніж є на складах. Виникає дефіцит.

- **Рішення:** Ми створюємо **фіктивного постачальника** A_{m+1} (додатковий рядок у таблиці).
- **Запас фіктивного постачальника:** $a_{m+1} = \sum b_j - \sum a_i$ (він віртуально покриває нестачу вантажу).
- **Тарифи:** Вартість перевезення від нього до будь-якого магазину теж дорівнює **нулю** ($c_{m+1,j} = 0$). Те, що в результаті розв'язку потрапить у цей рядок, покаже, які саме магазини залишаться без товару і на скільки одиниць.

4.1 Метод північно-західного кута

Уяви транспортну матрицю як звичайний двовимірний масив. **Метод північно-західного кута** — це найпростіший, "сліпий" алгоритм проходження цієї матриці. Він взагалі не дивиться на тарифи (вартість перевезень), а просто заповнює клітинки, рухаючись від верхнього лівого кута (індекс `[0][0]`) до правого нижнього.

Алгоритм крок за кроком:

Крок 1. Старт

Починаємо завжди з верхньої лівої клітинки таблиці — це і є наш "північно-західний кут" (постачальник 1, споживач 1).

Крок 2. Аналіз запасів та потреб

Дивимося на дві цифри:

- Скільки товару є на цьому складі (залишок у рядку).
- Скільки товару просить цей магазин (залишок у стовпці).

Крок 3. Заповнення клітинки

Ми завантажуюмо в цю клітинку **максимально можливу кількість товару**, тобто беремо мінімум із цих двох чисел:

```
Вантаж = min(Запас, Потреба)
```

Крок 4. Оновлення залишків і викреслювання

- Віднімаємо це завантажене значення від запасів складу та потреб магазину.
- Обов'язково хтось із них обнулиться (або склад спорожніє, або магазин отримає все).
- Якщо обнулився склад (рядок) — ми його **викреслюємо** і переходимо на одну клітинку **вниз** (до наступного складу).
- Якщо обнулився магазин (стовпець) — ми його **викреслюємо** і переходимо на одну клітинку **вправо** (до наступного магазину).

Крок 5. Повторення

У новій клітинці знову дивимося на актуальні залишки і повторюємо процедуру, малюючи таку собі "сходінку", поки не дійдемо до правого нижнього кута і не розподілимо всі нулі.

4.2 Метод мінімального елемента

Якщо метод північно-західного кута був «сліпим», то цей алгоритм — «жадібний». Він намагається на кожному кроці зекономити якнайбільше грошей. Замість того, щоб іти по порядку, він шукає найдешевші тарифи в усій таблиці.

Завдяки цьому ми отримуємо початковий план, який вже є досить непоганим, і метод потенціалів потім витратить набагато менше ітерацій на його покращення.

Алгоритм крок за кроком:

Крок 1. Пошук найдешевшого маршруту

Ми скануємо **всю** таблицю тарифів (вартостей перевезення c_{ij}) і знаходимо клітинку з **найменшим числом**.

(Що робити, якщо таких клітинок кілька однакових? Можна брати будь-яку, але зазвичай вибирають ту, куди можна завантажити більше товару).

Крок 2. Заповнення клітинки (максимальне завантаження)

Як і в попередньому методі, ми дивимося на запаси цього складу і потреби цього магазину. Беремо мінімум із цих двох чисел і записуємо в клітинку:

Вантаж = $\min(\text{Запас}, \text{Потреба})$

Крок 3. Оновлення залишків і викреслювання

- Віднімаємо записане число від запасів складу та потреб магазину.
- Якщо обнулився склад — **викреслюємо весь цей рядок** (більше з нього нічого не повеземо, навіть якщо там є інші дешеві тарифи).
- Якщо обнулився магазин — **викреслюємо весь цей стовпець** (йому більше нічого не треба).

(Якщо обнулилися одночасно і склад, і магазин, ми викреслюємо щось одне — або рядок, або стовпець, а іншому залишаємо "0", щоб не порушити правило $m + n - 1$ заповнених клітинок).

Крок 4. Повторення для залишків

Тепер ми дивимося тільки на ті клітинки, які **не викреслені**. Знову шукаємо серед них найменший тариф, знову вантажимо туди максимум можливого і викреслюємо рядок чи стовпець.

Процес триває, поки всі склади не спорожніють, а всі магазини не будуть задоволені.

4.3 Метод Фогеля

Якщо метод мінімального елемента був просто "жадібним", то метод Фогеля — **стратегічний**. Він дивиться на крок уперед і оцінює ризики. Метод рахує "штрафи": скільки грошей ми переплатимо, якщо випадково не займемо найдешевшу клітинку в рядку або стовпці, і нам доведеться брати наступну за дешевизною.

Цей метод дає настільки хороший початковий план, що часто він **відразу виявляється оптимальним** (або потребує мінімум ітерацій методу потенціалів для покращення).

Крок 2. Вибір максимального штрафу

Ми дивимося на всі обчислені штрафи (і рядкові, і стовпцеві) та вибираємо серед них **найбільший**.

Логіка: ми маємо в першу чергу закрити той напрямок, де ризик переплати найбільший. (Якщо є кілька однакових максимальних штрафів — вибираємо будь-який на свій смак, або той, де можна перевезти більше вантажу).

Крок 3. Заповнення клітинки

Ми вибрали рядок або стовпець із найбільшим штрафом. Тепер у цьому обраному рядку/стовпці ми знаходимо клітинку з **найменшим тарифом** і завантажуюємо туди товар по максимуму (як і в попередніх методах, беремо мінімум між залишком на складі та потребою магазину).

Крок 1. Обчислення штрафів (різниць)

Для **кожного** невикресленого рядка і **кожного** невикресленого стовпця ми робимо таку процедуру:

1. Знаходимо **два найменших тарифи** c_{ij} у цьому рядку (або стовпці).
2. Віднімаємо від більшого менший (тобто знаходимо їхню абсолютну різницю).

Ця різниця — це наш "штраф". Він показує: якщо ми зараз не завантажимо товар у найдешевшу клітинку, наступна найдешевша обійдеться нам на стільки-то гривень дорожче.

Ці штрафи зазвичай записують олівцем збоку від таблиці (біля рядків) і знизу (під стовпцями).

Крок 4. Викреслювання та оновлення

- Віднімаємо вантаж від запасів складу та потреб магазину.
- Викреслюємо рядок, який спорожнів, або стовець, який задовольнив потребу (якщо обнулилися обидва — викреслюємо щось одне, щоб зберегти умову $m + n - 1$ заповнених клітинок).

Крок 5. ПЕРЕРАХУНОК штрафів (Дуже важливо!)

Після викреслювання матриця змінилася. Тому ми **заново** рахуємо штрафи для всіх рядків і стовпців, що залишилися невикресленими. (Там, де нічого не викреслилось, штрафи можуть лишитися старими, але їх треба перевірити).

Повторюємо Кроки 2–5 доти, поки всі склади не спорожніють, а всі магазини не отримають товар.

4.4 Метод потенціалів

Переходимо до **Методу потенціалів**. Це основний алгоритм, який перетворює наш початковий (стартовий) план на ідеальний (оптимальний).

Уяви, що ми вже розподілили вантаж одним із попередніх методів (наприклад, мінімального елемента). Тепер ми маємо перевірити: а чи не можна перекинути частину вантажу іншим маршрутом, щоб загальна вартість стала ще меншою?

Крок 0. Перевірка на виродженість (ОБОВ'ЯЗКОВО!)

Перед початком алгоритму рахуємо кількість заповнених (базисних) клітинок у нашій таблиці. Їх має бути рівно $m + n - 1$ (де m — кількість рядків, n — стовпців).

Якщо заповнених клітинок менше, план називається **виродженим**. У такому випадку ми маємо штучно поставити цифру 0 у будь-яку порожню клітинку (так, щоб вона не утворювала замкненого циклу з іншими заповненими клітинками). Тепер ця клітинка з нулем вважається "заповненою", і ми можемо рахувати далі.

Крок 1. Знаходження потенціалів рядків і стовпців

Кожному рядку ми призначаємо потенціал U_i , а кожному стовпцю — потенціал V_j .

Ми шукаємо їх, використовуючи **ТІЛЬКИ ЗАПОВНЕНІ** клітинки. Для кожної заповненої клітинки має виконуватись рівняння:

$$U_i + V_j = c_{ij}$$

(де c_{ij} — це тариф на перевезення в цій клітинці).

Як це рахувати на практиці?

1. Ми маємо систему рівнянь, але невідомих на одне більше, ніж рівнянь. Тому ми беремо перший рядок і просто **прирівнюємо його потенціал до нуля**: $U_1 = 0$.
2. Далі, як по ланцюжку (або розгадуючи sudoku), знаходимо інші потенціали.
Наприклад, якщо в клітинці (1, 2) є вантаж, і її тариф $c_{12} = 5$, то: $U_1 + V_2 = 5 \implies 0 + V_2 = 5 \implies V_2 = 5$.
3. Знаходимо так усі U_i та V_j і записуємо їх збоку та знизу таблиці.

Крок 2. Оцінка порожніх клітинок (Дельти)

Тепер ми перевіряємо всі **ПОРОЖНІ** клітинки (де немає вантажу). Для кожної з них рахуємо оцінку Δ_{ij} (дельту) за формулою:

$$\Delta_{ij} = U_i + V_j - c_{ij}$$

(Тобто: потенціал рядка + потенціал стовпця - тариф самої клітинки).

Крок 3. Умова оптимальності

Дивимося на всі пораховані дельти:

- Якщо для **всіх** порожніх клітинок $\Delta_{ij} \leq 0$, вітаю! Наш план ідеальний, мінімум витрат знайдено.
- Якщо є хоча б одна клітинка з **додатною дельтою** ($\Delta > 0$), план можна покращити. Ми вибираємо клітинку з **найбільшою** додатною дельтою. Саме в неї ми будемо перекидати вантаж.

Крок 4. Побудова циклу перерахунку

Ми вибрали порожню клітинку (назвемо її "перспективною"). Тепер для неї треба побудувати цикл.

- **Цикл** — це замкнена ламана лінія.
- Вона починається і закінчується в нашій "перспективній" порожній клітинці.
- Всі **інші вершини (кути)** цього циклу повинні лежати **СТРОГО в заповнених клітинках**.
- Лінії циклу можуть йти тільки по горизонталі і вертикалі (повертати можна тільки під кутом 90 градусів, по діагоналі ходити не можна). Цикл може перетинати сам себе, це нормально.

Крок 5. Перекидання вантажу (Зсув по циклу)

1. Розставляємо у вершинах (кутах) циклу знаки «+» та «-», чергуючи їх. У нашій початковій (порожній) клітинці завжди ставимо «+», у наступній по циклу «-», потім знову «+», і так далі.
2. Дивимося тільки на ті вершини циклу, де стоїть знак «-». Шукаємо серед них клітинку з **найменшим** обсягом вантажу.
3. Це найменше число — і є той обсяг товару θ , який ми будемо перекидати.
4. Тепер перераховуємо цикл: у всіх клітинках з «+» ми **додаємо** це число θ , а у всіх клітинках з «-» — **віднімаємо** його.

Результат: Одна з клітинок (та, де був найменший мінус) обнулиться і стане порожньою, а наша перспективна клітинка заповниться. Ми отримали новий, дешевший план.

Після цього ми повертаємося до Кроку 1: наново рахуємо потенціали U та V для нового плану, наново рахуємо дельти для порожніх клітинок, поки всі дельти не стануть ≤ 0 .

Чому ми прирівнюємо одну змінну до нуля?

Уяви нашу таблицю. У нас є m рядків (це потенціали $U_1, U_2, \dots$) і n стовпців (це потенціали $V_1, V_2, \dots$).

Загальна кількість невідомих змінних, які нам треба знайти, дорівнює $m + n$.

Але ми маємо право складати рівняння ($U_i + V_j = c_{ij}$) **ТІЛЬКИ** для заповнених (базисних) клітинок. А їх у правильному плані завжди рівно $m + n - 1$.

Виходить класична ситуація з лінійної алгебри: **рівнянь на одне менше, ніж невідомих**.

Така система має безліч розв'язків. Щоб знайти хоча б один конкретний, ми маємо одну з невідомих зафіксувати — зробити її "вільною". Для зручності розрахунків завжди беруть перший потенціал рядка і кажуть: **Нехай $U_1 = 0$** . З цього нуля миттєво запускається ланцюгова реакція.

5. НЕЛІНІЙНЕ ПРОГРАМУВАННЯ

Достатня умова екстремуму (Матриця Гессе)

Гradient дорівнює нулю на вершині пагорба (максимум), на дні ями (мінімум) і в "сідлі" (як сідло для коня: в один бік іде підйом, в інший — спуск).

Як зрозуміти, що саме ми знайшли? Для цього потрібні **другі похідні**, які показують "кривизну" графіка. З них складають квадратну матрицю — **матрицю Гессе (Гессіан)** $H(x)$.

Ми рахуємо матрицю Гессе в нашій точці і перевіряємо її за критерієм Сильвестра:

1. Якщо матриця **додатно визначена** (головні мінори > 0) $\rightarrow$ функція вигнута вниз $\cup$.
Це **мінімум**.
2. Якщо матриця **від'ємно визначена** (мінори чергують знаки $-, +, -, \dots$) $\rightarrow$ функція вигнута вгору $\cap$. Це **максимум**.
3. Якщо матриця **знакозмінна** $\rightarrow$ це сідлова точка (ні мінімум, ні максимум).

5.1 мінімізація ф-цій однієї змінної

Тут ми шукаємо мінімум функції, яка залежить лише від однієї змінної: $f(x) \rightarrow \min$. Усі методи (дихотомії, золотого перетину, Фібоначчі) вимагають, щоб функція на заданому відрізку $[a, b]$ була **унімодальною**.

Унімодальна функція — це функція, яка на заданому відрізку має лише один мінімум (графік спочатку йде тільки вниз, досягає дна, а потім іде тільки вгору, як чаша).

Метод дихотомії (поділу відрізка навпіл)

Це найпростіший, класичний алгоритм. Його суть полягає в тому, щоб на кожному кроці відкидати ту половину відрізка, де мінімуму точно немає, поступово звужуючи зону пошуку, поки вона не стане меншою за задану точність ϵ .

Чому для пошуку мінімуму потрібні ДВІ точки?

Якби ми шукали корінь рівняння (де графік перетинає нуль), нам би вистачило однієї точки рівно посередині відрізка. Але ми шукаємо мінімум! Якщо ми просто візьмемо одну точку посередині і знайдемо її значення $f(x_c)$, це число нічого нам не скаже. Ми не будемо знати, чи графік у цій точці йде вниз, чи вже піднімається вгору.

Тому в методі дихотомії ми беремо **дві точки**, розташовані дуже близько одна до одної поблизу центру відрізка. Порівнюючи їхні значення, ми визначаємо "нахил" (тренд) функції.

Алгоритм методу дихотомії (крок за кроком):

Початкові дані: * Відрізок $[a, b]$.

- Задана точність ϵ (наприклад, 0.1).
- Константа розрізнення δ — це дуже маленьке число (набагато менше за ϵ), яке показує відстань між двома нашими центральними точками.

Крок 1. Знаходження центральних точок

На кожній ітерації ми обчислюємо дві точки x_1 та x_2 , симетрично відступивши від абсолютної середини відрізка на мікрокрок $\frac{\delta}{2}$:

$$x_1 = \frac{a+b}{2} - \frac{\delta}{2}$$

$$x_2 = \frac{a+b}{2} + \frac{\delta}{2}$$

Крок 2. Обчислення значень функції

Рахуємо значення нашої цільової функції в цих двох точках: $f(x_1)$ та $f(x_2)$.

Крок 3. Порівняння та відкидання (Звуження відрізка)

Ми порівнюємо ці два значення, щоб зрозуміти, куди йде графік:

1. Якщо $f(x_1) < f(x_2)$:

Це означає, що права точка лежить вище за ліву. Графік зростає в правий бік.

Оскільки функція унімодальна (чаша), дно цієї чаші гарантовано знаходиться **лівіше** від точки x_2 .

Тому праву частину відрізка (від x_2 до b) ми сміливо **відкидаємо**.

Новий відрізок: $[a, x_2]$.

2. Якщо $f(x_1) > f(x_2)$:

Ліва точка лежить вище за праву. Графік спадає в правий бік (або зростає в лівий).

Мінімум гарантовано знаходиться **правіше** від точки x_1 .

Ліву частину відрізка (від a до x_1) ми **відкидаємо**.

Новий відрізок: $[x_1, b]$.

3. Якщо $f(x_1) = f(x_2)$:

Нам дуже пощастило, ми потрапили рівно на протилежні стінки чаші на однаковій висоті. Мінімум лежить строго між ними.

Новий відрізок: $[x_1, x_2]$.

Крок 4. Перевірка на зупинку

Ми перевіряємо довжину нашого нового звуженого відрізка. Якщо його довжина стала меншою або рівною нашій заданій точності:

$$b - a \leq \epsilon$$

Тоді алгоритм зупиняється. За координату мінімуму (відповідь) беруть середину цього фінального маленького відрізка: $x^* = \frac{a+b}{2}$.

Якщо ж відрізок усе ще з великий, ми повертаємося до Кроку 1 і знову ділимо його навпіл.